Spotter AI: Freight Rate Prediction

A machine learning system that predicts truckload freight rates. I built it for a technical assessment, using CatBoost regression and validating chronologically so the model is always tested on data that comes after what it trained on.

Table of Contents

1. Project Overview
2. Dataset Understanding
3. Feature Engineering
4. Validation Strategy
5. Model Experimentation and Selection
6. Error Analysis and Limitations
7. Final Model Training
8. Prediction and Inference
9. Installation and Usage
10. Output Validation
11. Conclusion

1. Project Overview

Business problem

Brokers and shippers need a good rate estimate before a load goes out to the market. A reliable pricing model cuts down on manual quoting, makes margins easier to predict, and lets a team react faster when the market moves.

Objective

The goal is to train a regression model on historical shipments to predict `posted_rate`, the rate actually paid for each shipment, and then use it to produce two sets of predictions:

- **12,000 unlabeled validation shipments** from January to October 2025
- **31 daily forecasts for December 2025** on one fixed lane, Lexington → Fort Wayne, Dry Van

Input datasets

File	Rows	Description
`data/train-test.csv`	48,000	Labeled shipments, January-October 2025
`data/validation.csv`	12,000	Unlabeled shipments, same period
`data/december-chart-inputs.csv`	31	Fixed route, December 2025, one row per day

Target variable

`posted_rate` is the freight rate paid for each shipment, in USD, and it is always positive. The model predicts it directly on its original scale.

Deliverables

- `validation_predictions.csv`: 12,000 rows with `load_id` and `predicted_rate`
- `data/december-chart-inputs.csv`: the original file with `predicted_rate` filled in
- `scorer_results/candidate_december.png`: the December rate trend chart

2. Dataset Understanding

Labeled training data (`data/train-test.csv`)

Shape: 48,000 rows × 14 columns

Column	Type	Description
`load_id`	string	Unique shipment identifier (e.g., `TR-000001`)
`pickup`	string	Origin city
`delivery`	string	Destination city
`pickup_lat`, `pickup_lon`	float	Pickup coordinates
`delivery_lat`, `delivery_lon`	float	Delivery coordinates
`distance`	float	Reported road distance in miles
`equipment`	string	Trailer type: Dry Van, Reefer, or Flatbed
`weight`	float	Shipment weight in pounds
`date`	date	Shipment date
`market_index`	float	External market signal
`quote_signal`	float	External quote signal
`posted_rate`	float	Actual freight rate paid (target)

Missing values:

Column	Missing
`weight`	300
`market_index`	374
All other columns	0

Target distribution (`posted_rate`):

Statistic	Value
Min	$57.22
25th percentile	$1,251.56
Median	$2,030.76
Mean	$2,373.98
75th percentile	$3,330.75
Max	$25,533.00

The distribution is right-skewed. Most shipments cost less than $5,000, and a thin tail of high-value loads stretches far beyond that.

Equipment distribution:

Equipment	Records
Dry Van	27,202 (56.7%)
Reefer	12,045 (25.1%)
Flatbed	8,753 (18.2%)

Route structure:

- 64 unique pickup cities and 64 unique delivery cities
- 4,014 unique routes
- A median of 10 records per route
- 758 routes with 5 or fewer records

Distance and rate correlation:

Distance is by far the strongest predictor, with a Pearson correlation of 0.91 against `posted_rate`. `market_index` and `quote_signal` each have almost no raw correlation with the target (around ±0.04).

High-value shipment analysis:

Segment	Records	Proportion
Below $5,000	45,735	95.28%
$5,000-$10,000	2,162	4.50%
$10,000-$15,000	79	0.16%
$15,000-$20,000	15	0.03%
Above $20,000	9	0.02%

Only 103 of the 48,000 shipments (0.215%) are above $10,000. They show up at a steady rate across months. October has 12 of them, or 0.247% of that month's records, which is close to the dataset-wide average.

Validation data (`data/validation.csv`)

12,000 rows with the same columns as the training data, minus `posted_rate`. Load IDs run from `TE-000001` to `TE-012000`.

December forecast data (`data/december-chart-inputs.csv`)

31 rows, one for each day of December 2025. Everything except the date is fixed:

Field	Value
pickup	Lexington
delivery	Fort Wayne
distance	360 miles
equipment	Dry Van
weight	32,000 lb
date	2025-12-01 to 2025-12-31

The task is to show how the rate on this one lane moves from day to day through the month.

3. Feature Engineering

All the feature engineering lives in `src/feature_engineering.py`. Training, validation, and December inference all go through the same `prepare_features()` function, so the pipeline can't drift between stages. It returns a fixed-order, 15-column DataFrame that is checked against `MODEL_FEATURES`.

The December file has no coordinates, so I left out geographic features such as Haversine distance. That keeps the model shared across all three datasets, and all 15 features exist in each of them.

The 15 model features

Categorical features (passed to CatBoost as category indices):

Feature	Description
`pickup`	Origin city, encoded as a string
`delivery`	Destination city, encoded as a string
`equipment`	Trailer type (Dry Van / Reefer / Flatbed)
`route`	Origin and destination joined together, e.g. `"Lexington_Fort Wayne"`
`distance_bin`	Distance tier (7 levels, from very_short to extreme)

Numeric features:

Feature	Description
`distance`	Reported road distance in miles
`weight`	Shipment weight in pounds (NaN is preserved)
`month`	Calendar month (1-12)
`day_of_week`	Day of week (0 = Monday, 6 = Sunday)
`day_of_year`	Day of year (1-366)
`is_weekend`	1 if Saturday or Sunday, otherwise 0
`day_sin`	Sine encoding of `day_of_year`: `sin(2π × d / 365.25)`
`day_cos`	Cosine encoding of `day_of_year`: `cos(2π × d / 365.25)`
`weight_missing`	1 if weight is missing, otherwise 0
`weight_per_mile`	`weight / distance` (NaN when weight is missing)

Distance bins

```
(-inf, 250]    → very_short
(250,  500]    → short
(500,  1000]   → medium
(1000, 1500]   → medium_long
(1500, 2000]   → long
(2000, 3000]   → very_long
(3000, +inf)   → extreme
```

Cyclical date encoding

Day-of-year is not cyclical on its own. Day 365 and day 1 sit at opposite ends of the number line even though they are one day apart. Sine and cosine encoding fixes that by wrapping the year onto a circle:

```
day_sin = sin(2π × day_of_year / 365.25)
day_cos = cos(2π × day_of_year / 365.25)
```

I keep both, because either one alone can't tell you where you are in the cycle.

Categorical feature handling

CatBoost deals with categorical features natively, using ordered target statistics. These are a form of target encoding computed over a random permutation of the training set, which stops the target from leaking into its own encoding. Every categorical column is cast to a string before it reaches the model, and missing categorical values become the sentinel string `"__MISSING__"`.

Missing numeric values

CatBoost handles missing numeric values without imputation, so `weight` stays `NaN` when it's absent, and `weight_per_mile` is `NaN` along with it. The `weight_missing` flag lets the model tell a missing weight apart from a weight of zero.

4. Validation Strategy

Why chronological splitting

Freight rates change over time. A shipment from October 15 reflects market conditions that a January shipment never saw. A random train-test split would let patterns from later periods leak into training, which gives scores that look better than anything the model could achieve on genuinely future data.

So every split uses a strict cutoff: the training set only contains records from before the validation window starts.

Four-window evaluation

Window	Training period	Validation period	Train rows	Val rows
July	Jan-Jun 2025	Jul 2025	28,806	4,912
August	Jan-Jul 2025	Aug 2025	33,718	4,759
September	Jan-Aug 2025	Sep 2025	38,477	4,670
October	Jan-Sep 2025	Oct 2025	43,147	4,853

Model comparison across all four windows

Model	Avg MAE	Avg RMSE	Avg R^2	Avg MAPE
Baseline 1: training median	$1,138.41	$1,550.11	−0.0568	69.57%
Baseline 2: distance-only linear	$193.69	$649.19	0.8146	9.73%
CatBoost 15-feature model	$119.00	$629.01	0.8260	5.59%

Compared with the distance-only linear baseline, CatBoost cuts average MAE by about **38.6%**. Compared with the constant median baseline, the cut is about **89.5%**.

The distance-only baseline is a useful yardstick because distance explains most of the variance in rates (correlation 0.91). What CatBoost adds on top of it comes from route identity, equipment type, weight, and the date features.

> **Note:** These numbers come from chronological validation on held-out months of the labeled data. They are not scores from an independent hidden test set. Spotter runs the final evaluation after submission.

5. Model Experimentation and Selection

Unless stated otherwise, every experiment used the same split: train on January-September, validate on October. I didn't use October labels to make any modelling decision until model selection was finished.

Experiments conducted

Experiment	Description	October MAE	Outcome
A	15-feature CatBoost baseline	$115.18	**Selected**
B	+ `market_index`, `quote_signal`	Higher	Rejected
C	+ log-transformed target	Higher	Rejected
D	+ 5 additional date/interaction features, depth 10, lr 0.03	$117.77	Rejected
E	Route historical median and mean	$119.00 avg	Rejected
F	Equipment-route historical median	Higher	Rejected
G	Route + equipment-route aggregates	Higher	Rejected
H	Aggregates + Haversine distance	Higher	Rejected

Route feature experiment detail (four windows)

Experiment	Avg MAE	vs Baseline A
A: 15-feature baseline	$119.00	n/a
B: + route median/mean	$121.74	+2.3%
E: + aggregates + Haversine	$130.74	+9.9%
C: + equipment-route median	$135.29	+13.7%
D: + route + equipment-route	$141.57	+19.0%

Adding route-level historical statistics by hand never beat the plain baseline. CatBoost's ordered categorical encoding already picks up category-level pricing on its own, and the extra aggregates mostly added noise, especially on routes with fewer than five historical records (758 routes in the training data).

Selected model: October validation results

Metric	Value
MAE	$115.18
RMSE	$647.80
R^2	0.8204
MAPE	5.70%
Best iteration	553 (selected_iterations = 554)
High-value MAE (>$10K)	$10,469.08

Final CatBoost configuration

Parameter	Value
iterations	1,500 (configured); 554 (early stopping)
depth	8
learning_rate	0.05
loss_function	MAE
eval_metric	MAE
early_stopping_rounds	100
random_seed	42

6. Error Analysis and Limitations

Equipment-wise performance (October)

Equipment	Records	MAE
Dry Van	~2,750	~$102
Reefer	~1,220	~$130
Flatbed	~880	~$134

Dry Van is the most common equipment type and has the lowest error. Reefer and Flatbed do worse, which fits with their smaller sample sizes and less predictable pricing.

Distance-wise error pattern

Errors grow with distance. Short hauls (under 500 miles) are predicted better than long hauls (over 1,500 miles), where rates vary more and each lane has less training data.

High-value shipment limitation

The model's biggest structural weakness is that it underpredicts rare, high-value shipments above $10,000. Those are 103 of the 48,000 training records (0.215%).

Examples from October validation:

Route	Actual rate	Predicted rate
Milwaukee → Bakersfield	$17,514	~$3,614
Columbia → Tucson	$17,893	~$4,230
Boston → Bakersfield	$19,110	~$5,869

There simply isn't much training signal for rates like these. I didn't remove them from the training data, since nothing suggests they're corrupt. Their share by month is steady (0.215% overall against 0.247% in October), which points to real market events and not data errors.

The overall MAE of $115.18 looks healthy, but it hides how poorly the model does on extreme rates — so be careful when a shipment is likely to price above $5,000.

What was not attempted

- Removing or downweighting high-value outliers, because there was no good evidence of a data problem
- Any external market data beyond the provided `market_index` and `quote_signal`, since both had near-zero raw correlation with the target and didn't help validation performance when I added them

7. Final Model Training

Once the experiments were done and the original 15-feature CatBoost setup had come out on top, I retrained the final model on all 48,000 labeled records (January-October 2025).

Why October is included in final training

October served as the validation set while I was choosing a model. With the configuration locked, retraining on the full labeled set gives the model another 4,853 records and exposes it to October market conditions. That should help it generalise to November and December. There's no leakage risk here, because no modelling decisions are made after this point.

Final training configuration

Parameter	Value
Training records	48,000
Iterations	554 (from experiment metrics)
Depth	8
Learning rate	0.05
Loss function	MAE
Random seed	42
Model artifact	`models/freight_rate_model.cbm`

The iteration count (554) is read from `reports/experiment_metrics.json`, which the experiment phase writes. `train_final()` raises `FileNotFoundError` if that file is missing.

> **Important:** Running `src/train.py` directly currently calls `train_final()`. To rerun the October experiment and regenerate `experiment_metrics.json`, switch the call to `train_experiment()` (see Installation and Usage).

8. Prediction and Inference

`src/predict.py` loads the saved final model and produces both required outputs.

Workflow

1. Load `models/freight_rate_model.cbm` with `CatBoostRegressor.load_model()`
2. Read `data/validation.csv` (12,000 rows)
3. Run `prepare_features()`, the same pipeline used in training
4. Align the feature columns to the model's expected order using `model.feature_names_`
5. Predict, then check that nothing is NaN, non-finite, or non-positive
6. Write `validation_predictions.csv` with columns `load_id`, `predicted_rate`
7. Read `data/december-chart-inputs.csv` (31 rows)
8. Apply the same feature pipeline and predict
9. Write the predictions into the existing `predicted_rate` column, leaving every original column untouched

Observed prediction ranges

Dataset	Min	Max
Validation (12,000 rows)	$187.36	$7,020.33
December (31 rows)	$838.55	$855.83

The December range is narrow because the route, equipment, distance, and weight never change. Only the date does, so the day-to-day wiggle comes from the seasonal and day-of-week patterns the model learned. These are predictions only. Actual December rates aren't available, so nothing has been checked against ground truth.

9. Installation and Usage

Requirements

- Python 3.10 or later
- Windows PowerShell (the commands below were tested on Windows)

`requirements.txt` lists:

```
matplotlib>=3.8,<4
numpy>=1.26,<3
pandas>=2.0,<3
```

CatBoost and scikit-learn are used by the project but aren't pinned in `requirements.txt`, so install them separately alongside the listed packages.

Setup

```
python -m venv venv
venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install catboost scikit-learn
```

Run the October experiment (optional, regenerates metrics and the experiment model)

You need to run this before final training if `reports/experiment_metrics.json` doesn't exist.

Open `src/train.py`, change the main block to call `train_experiment()`, then run this from the project root:

```
python src/train.py
```

It trains on January-September, evaluates on October, and writes:

- `reports/experiment_metrics.json`
- `reports/october_predictions.csv`
- `models/experiment_model.cbm`

Run final training

Put the main block back to calling `train_final()` (the current default), then run:

```
python src/train.py
```

This trains on all 48,000 records for 554 iterations and writes `models/freight_rate_model.cbm`.

Generate predictions

```
python src/predict.py
```

Writes:

- `validation_predictions.csv`
- `data/december-chart-inputs.csv` (with `predicted_rate` filled in)

Run the official scorer

```
python score.py --predictions validation_predictions.csv --december-predictions
data/december-chart-inputs.csv
```

Expected output:

```
Validated 12,000 final predictions.
Validated 31 fixed December predictions.
Created chart: scorer_results\candidate_december.png
Final validation metrics are calculated by Spotter after submission.
```

Run baseline and route-feature experiments (optional)

```
python experiments/baseline_comparison.py
python experiments/route_features_experiment.py
```

Both scripts write CSV reports to `reports/`. Expect each to take 20 to 40 minutes, depending on your hardware.

Run error analysis on October predictions

```
python src/evaluate.py
```

To rerun the CatBoost variant comparison:

```
python experiments/compare_models.py
```

This reads `reports/october_predictions.csv` and prints breakdowns by equipment, by distance, and by largest error.

Troubleshooting

`ModuleNotFoundError: No module named 'feature_engineering'`

Run every script from the project root, not from inside `src/`:

```
# Correct
python src/train.py

# Incorrect
cd src
python train.py
```

`FileNotFoundError: Experiment metrics not found`

`reports/experiment_metrics.json` doesn't exist yet. Run the experiment phase first by calling `train_experiment()` in `src/train.py`.

`ModuleNotFoundError: No module named 'catboost'`

Install it with `python -m pip install catboost`.

`ERROR: predictions must contain exactly two columns`

The scorer wants `load_id,predicted_rate` and nothing else, with no index column. Make sure `validation_predictions.csv` came straight from `src/predict.py` and hasn't been edited.

10. Output Validation

The official scorer (`score.py`) checks both output files before it draws the December chart.

Validation predictions (`validation_predictions.csv`)

Requirement	Value
Exact row count	12,000
Columns (in order)	`load_id`, `predicted_rate`
Load ID format	`TE-000001` through `TE-012000`
No duplicates or missing IDs	Required
All `predicted_rate` values	Finite and positive

December predictions (`data/december-chart-inputs.csv`)

Requirement	Value
Exact row count	31
Columns (in order)	`pickup`, `delivery`, `distance`, `equipment`, `weight`, `date`, `predicted_rate`
Dates	2025-12-01 through 2025-12-31, no duplicates
Fixed fields unchanged	pickup = Lexington, delivery = Fort Wayne, distance = 360, equipment = Dry Van, weight = 32,000
All `predicted_rate` values	Finite and positive

What the scorer does and does not verify

Passing the scorer tells you both files are formatted correctly and complete. It does not measure predictive accuracy. Spotter calculates the final evaluation metrics using withheld ground truth labels.

11. Conclusion

This project delivers a post-production-ready freight rate prediction model trained on 48,000 historical shipments using CatBoost regression.

The model was selected through systematic experimentation across eight feature and modelling variants, evaluated on four chronological validation windows rather than a single split. The original 15-feature configuration outperformed or matched every alternative tested, including additional market signals, log-transformed targets, expanded date features, route-level historical aggregates, and geographic features.

Across four chronological windows, the CatBoost model achieved an average MAE of $119.00, a 38.6% reduction over a distance-only linear baseline and an 89.5% reduction over a constant median predictor. On the October held-out window specifically, the model achieved MAE $115.18, RMSE $647.80, and R^2 0.8204.

The primary known limitation is underprediction of rare high-value shipments (above $10,000). These represent 0.215% of the training data, and their sparse occurrence limits what any supervised model trained on this dataset can learn about them without additional data.

Both required prediction files have been generated and verified against the official scorer. The full experiment log is preserved in `reports/` to document that model selection was based on measured validation performance.

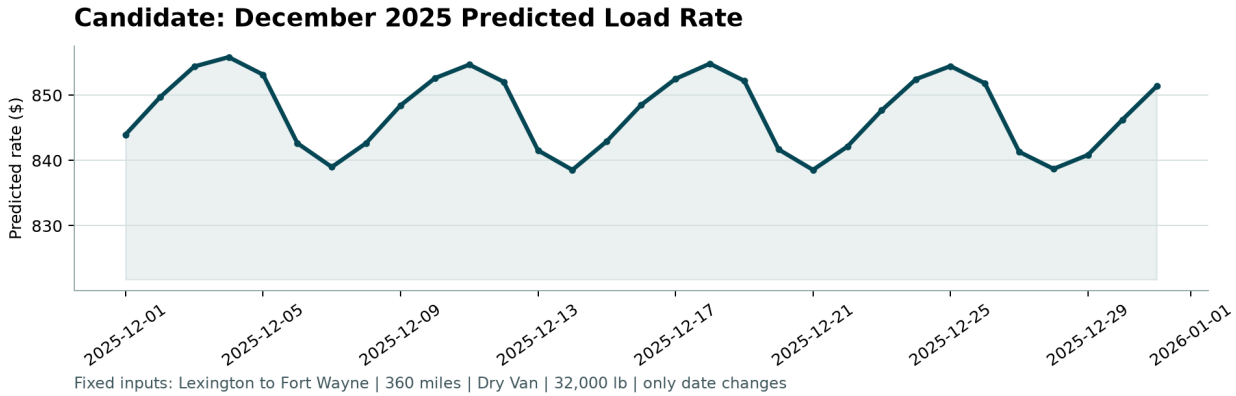


Figure: Predicted daily load rate for December 2025 (Lexington to Fort Wayne, Dry Van, 32,000 lb).

Potential future improvements:

- Additional training data covering more high-value shipments
- Separate model or post-processing adjustment for the high-value segment
- Lane-level time-series features built from a larger historical dataset
- Retraining on a rolling basis as new market data becomes available